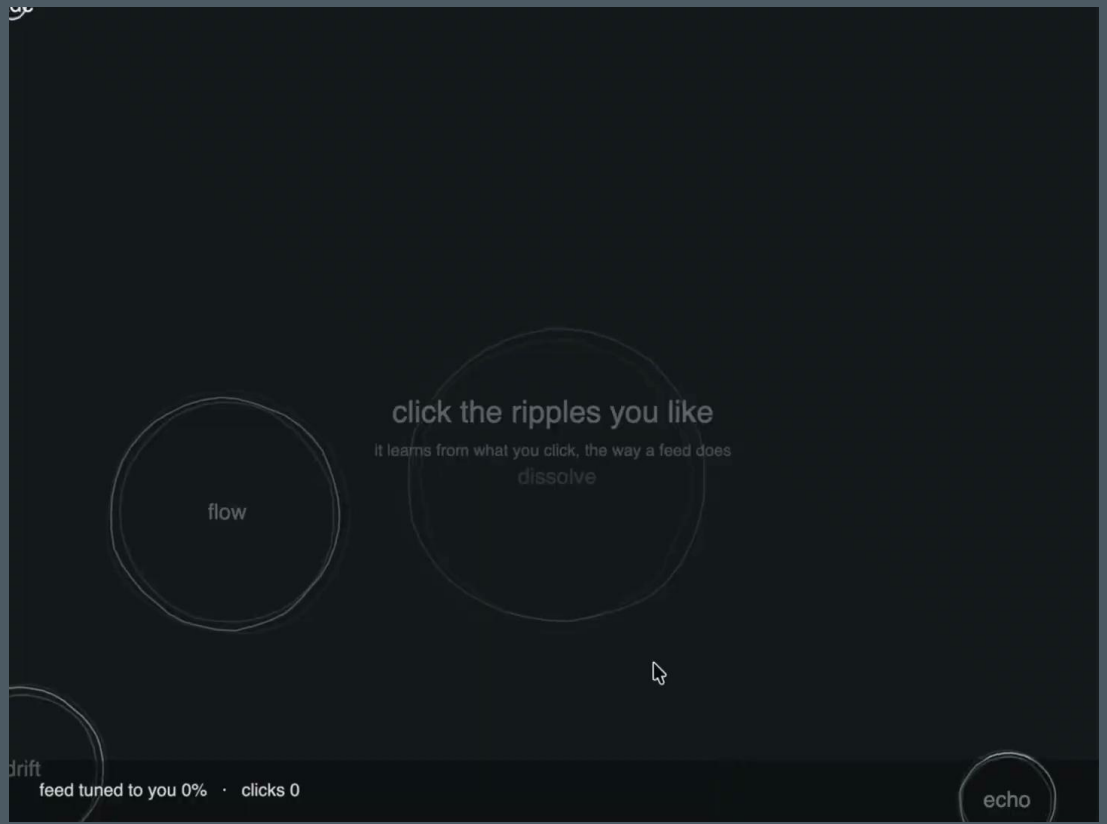


Dhatri Jayanth

FINAL DELIVERABLES

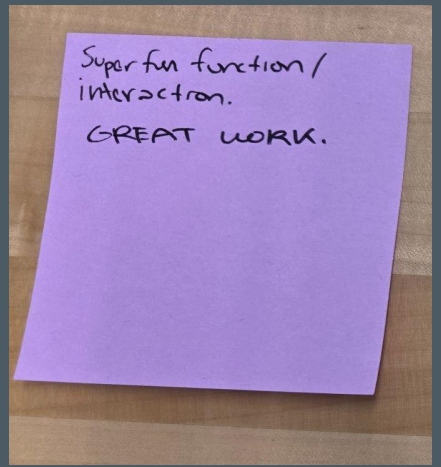
Ripple

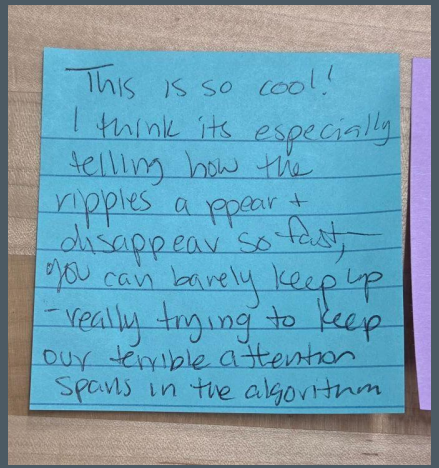
FINAL PRODUCT

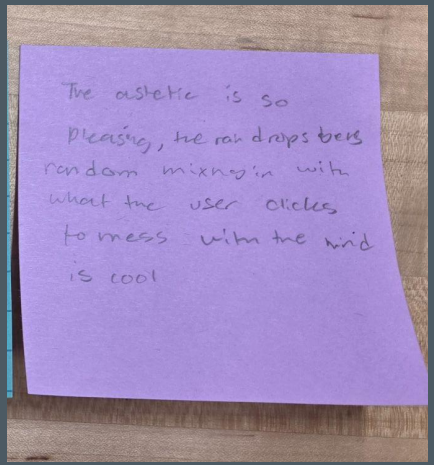


This piece shows a simple truth: a system stuck inside its own rules can only shuffle what it's given, not create anything new, until something from outside interrupts it.

Feedback



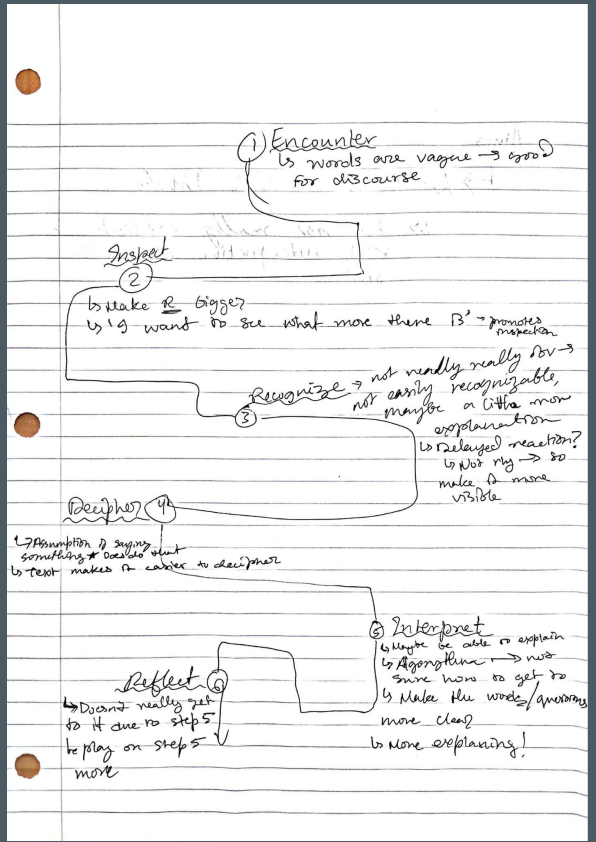


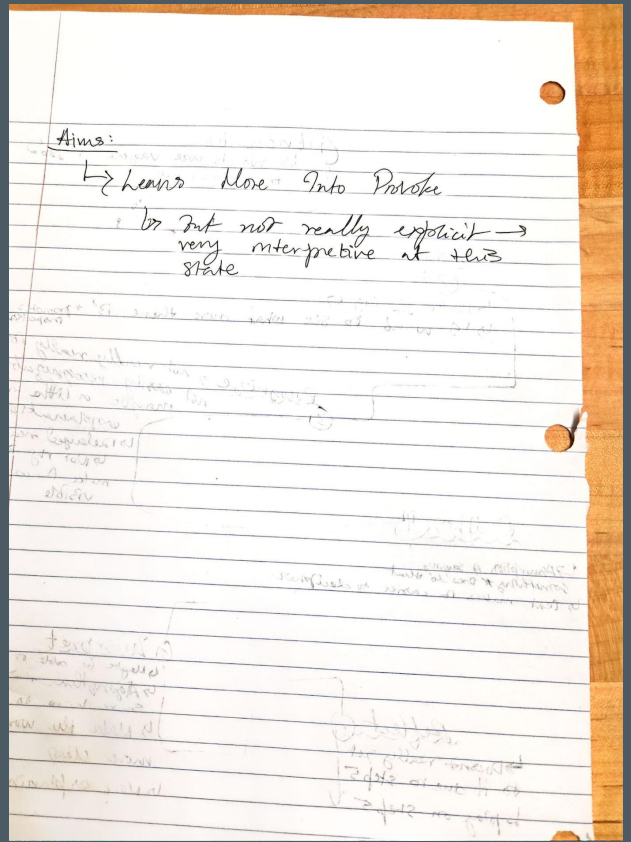


I only got three sticky notes, so I assume that not a lot of people got to my computer, or that I did not explain my concept well. The feedback that I also got was on my 'aesthetic' not the substance of the experience.

One of those notes called the vagueness 'good for discourse' without using that term. That's a discursive design strategy working exactly as intended: the ambiguity gave people room to read their own meaning into it, instead of being handed one.

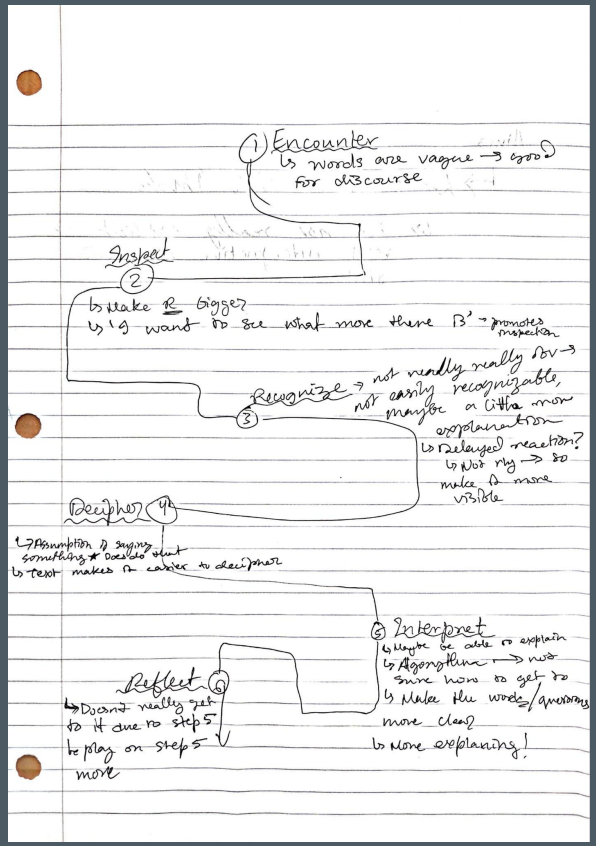
Feedback


1 Encounter
↳ words are vague → good for discourse
Inspect
2
↳ Make R bigger
↳ I want to see what more there is' → promotes inspection
Recognize → not readily really fav → not easily recognizable, maybe a little more explanation
3
↳ Delayed reaction?
↳ Not my → so make it more visible
Decipher 4
↳ Assumption I assume something + possible intent
↳ text makes it easier to decipher
5 Interpret
↳ Maybe be able to explain
↳ Agonistic → not sure how to get to
↳ Make the words/ awareness more clear
↳ More explaining!
Reflect 6
↳ Doesn't really get to it due to step 5
↳ play on step 5 more


Aims:
↳ Learn More Into Provoke
↳ But not really explicit → very interpretive at this state

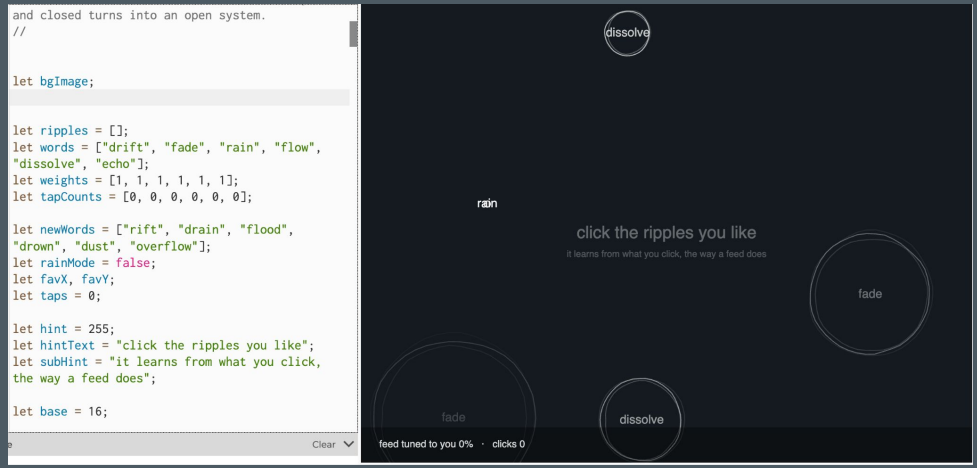
Feedback

The longer conversation mapped the experience into six stages: encounter, inspect, recognize, decipher, interpret, reflect. Two problems surfaced inside that map. First, on ‘encounter,’ people called the words vague, and meant it as praise: ambiguity gave them room to project their own reading onto the piece. But that same vagueness broke ‘recognize,’ the stage where someone is supposed to notice a mechanism acting on them specifically. Nobody could tell the ripples were responding to their individual clicks rather than running on a generic timer, so there was nothing concrete to decipher, and the process stalled before it reached interpretation. Second, and more basic: the ‘press R’ prompt that triggers the shift into weather mode was small, gray, and tucked into a corner. Almost nobody found it. That means almost nobody reached the half of the piece that actually makes the argument, the part where control passes from the user to something outside the system.



The Ripple

Ripple is a pond on screen that rains words: drift, fade, rain, flow, dissolve, echo. Click a word, and the system does two things at once. It raises that word's weight in a pool of six, so it is more likely to be picked again, and it nudges the point where new ripples land closer to wherever you just clicked. Do this enough times (the code tracks it as a 0 to 1 personalization value, climbing with every seventh click) and the pond stops scattering words randomly across the screen. It starts clustering them around one point, saying the same word back to you more and more often, until there is nothing left to discover. That is the entire mechanism of a recommendation algorithm, reduced to six words and one variable: a system that learns what you touch, and narrows what it shows you in response. The pond is simulating a feed or algorithm.



You click on words the way you'd click on a post, and watch the pond slowly learn what you like, the same way a feed does. The point is to feel that happen, not read about it afterward.

DESIGN PROCESS

I chose the ripple specifically because its weakest moment doubled as its clearest argument. Its early version dropped a new word every forty frames at a random screen position, picked from a fixed list, on a strict interval, like a metronome. That repetitiveness read as a flaw when I first built it, a kind of mechanical sameness the plant garden and the sundial did not have. But a closed system imitating an open one is supposed to feel mechanical underneath its surface.

That's a discursive choice as much as an aesthetic one. I picked the sketch that was honest about its own limits over the two that looked more finished, because the goal was to get a reaction out of someone, something the polished sketches never would have done.

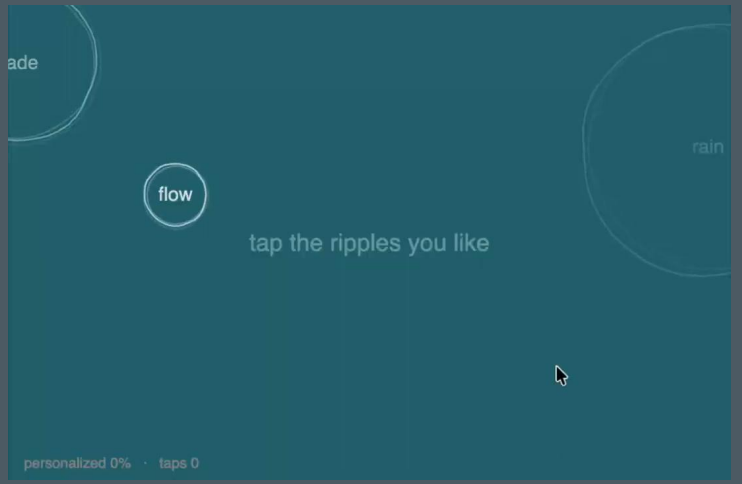


DRAFT

My first instinct was to place the ripples inside a scene: a pond with a horizon, something with depth you would look into, the way you'd look at a photograph. Editing that first draft, I kept stripping away anything that asked to be looked at once and understood.

Then, I started with a flat teal background meant to read as sky reflected in water. In practice it read as decoration: a color choice with no argument behind it. I moved to a near black backdrop instead, closer to what a pond actually looks like at night, where you cannot see the bottom. That opacity fits the subject. You cannot see into a personalization algorithm's internals either; you only see what it surfaces. Our class session with Tega Brain pushed the concept further, toward the idea that ended up anchoring the whole project: tying the sketch's behavior to real, live weather data over Austin. Before that session, 'randomness' in the sketch meant JavaScript Math.random, a formula with no connection to anything outside its own code. Tying it to actual wind speed and pressure meant the system's behavior could finally be shaped by something it did not generate itself.





DRAFT

A pattern can show its own process: words appearing, clustering, repeating, which is the only way to make a learning system visible rather than described. Once I saw that distinction, the pond came to be just a plain black screen. I also experimented with adding a background, which felt a little too crowded.



DESIGN PROCESS

Four changes came directly out of the critique, each answering a specific note:

For the recognition problem, the opening line now pairs with a second, quieter line underneath it, ‘it learns from what you click, the way a feed does,’ so the mechanism is stated rather than left for the viewer to infer.

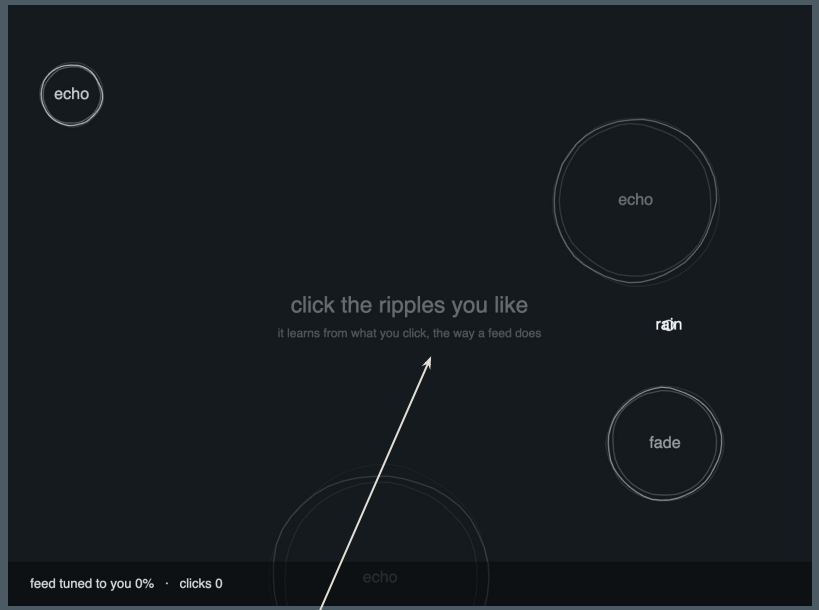
For the visibility problem, the ‘press R’ prompt no longer sits small and gray in a corner; it now appears centered, larger, and pulsing once personalization crosses ninety percent, so viewers actually reach the shift into weather mode.

For the general note that information on screen was too faint to register as information, the entire HUD now sits on a dark, semi transparent bar instead of floating as low contrast gray text, so it reads as a readout rather than a decorative afterthought.

And for the request to connect the piece more explicitly to algorithmic systems, the sketch now labels its own output: once your clicks have pulled the cluster point far enough, it tags new words near that point ‘recommended for you,’ borrowing the exact language a real feed uses to describe what it is actually doing to you.

Each of these changes is really about calibration: how much clarity a discursive piece needs before the ambiguity people liked stops working as discourse and starts reading as confusion instead.

THE CHANGES

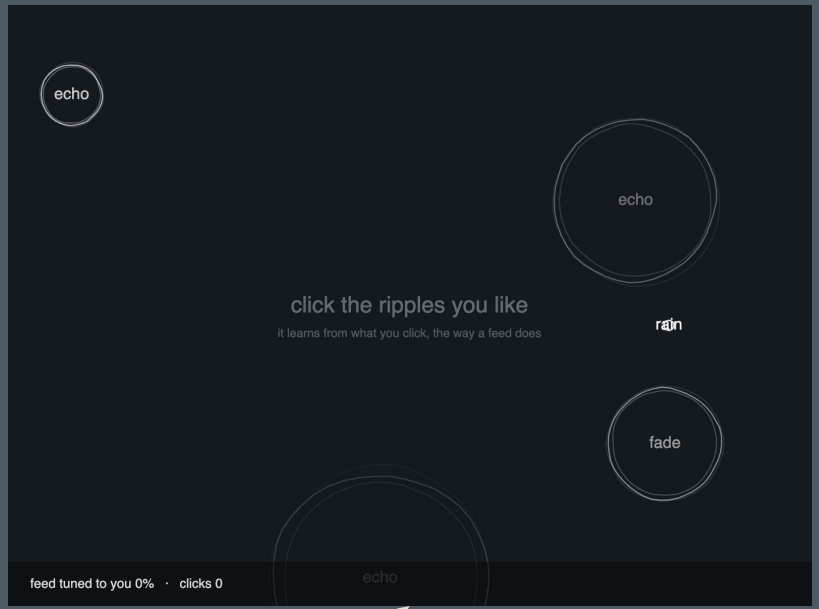


First Change



Second Change

THE CHANGES



Third Change

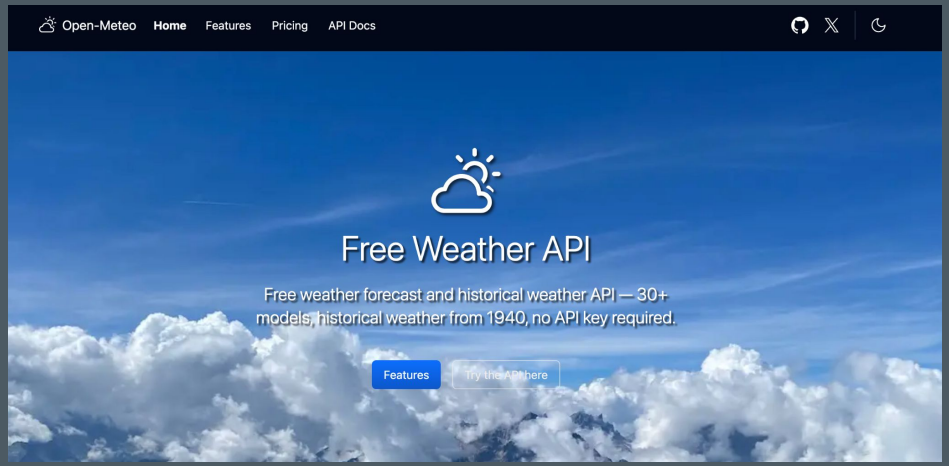


Fourth Change

PAIN POINT

It would have been easy to just print the wind speed as a label on screen, data as decoration, cosmetic rather than structural. Instead, wind speed now sets the interval between ripples, precipitation compresses that interval further when it is actually raining, and both wind speed and pressure together seed the sketch's random number generator directly through randomSeed. That last part matters most. It means the sequence of words falling in rain mode is no longer determined by anything inside the sketch's own code. It is determined by the sky.

Everything before connecting the API described a closed system. This moment, where the weather takes over, is the part designed to make that system feel open instead.




```
function getWeather() {
  let url = "https://api.open-
meteo.com/v1/forecast" +
            "?
latitude=30.2672&longitude=-97.7431" +

"&current=wind_speed_10m,surface_pressure,precip
itation";
  loadJSON(url, gotWeather);
  lastFetch = frameCount;
}
```

REFLECTION

I made this hoping it would actually rain in Austin, and the words falling on screen would match the weather by coincidence. They never did. The sketch was just picking random words on a timer, nothing was really connected to the sky. That's the point Bridle makes: a computer can only shuffle around what you already gave it, it can't bring in anything truly new on its own. When I showed people the first version, nobody noticed that clicking a word taught the sketch to show it more. They just said it looked nice. So I went back and made that part obvious: a line that says it learns from your clicks, a bigger prompt for the moment weather takes over, and a label that says "recommended for you" once it's picked up on your pattern. I also made the background plain black, since there's nothing behind this system except its own rules, and I didn't want a pretty photo hiding that. None of this changed how the sketch actually works. It just made the trick easier to see.

What I hope for this project is that showing the mechanism would say more than explaining it ever could, and that people would sit with it a little before moving on.